



Team Members

Harindham Sharma

Aman Kumar

Ananya Agarwal

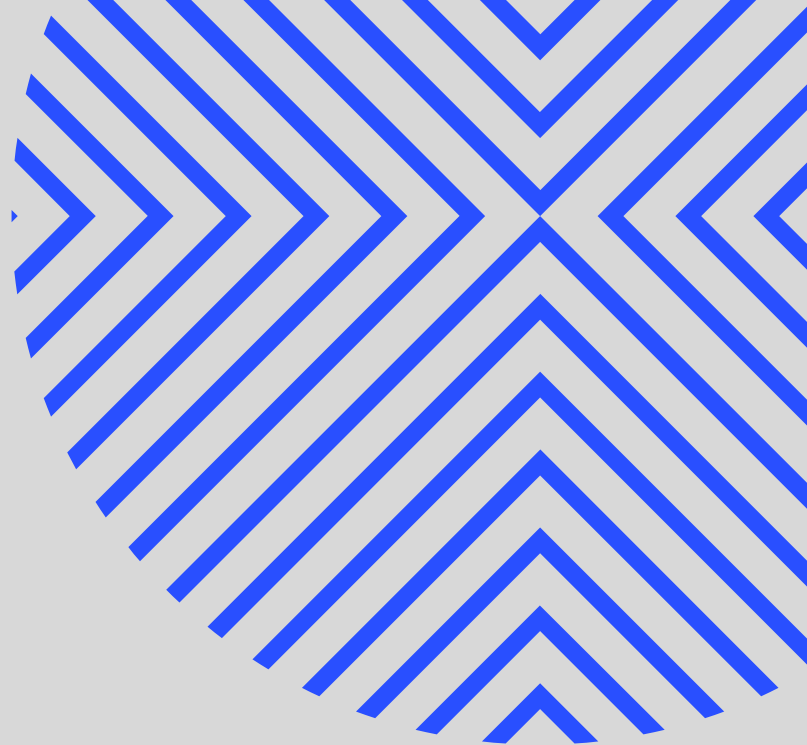
Gurleen Kaur

Aditya Kolluru Naidu

BIG DATA CSGY 6513

ANOMALYZE

TABLE OF CONTENTS



01 | PROBLEM
STATEMENT

02 | MOTIVATION

03 | SOLUTION

04 | SYSTEM
ARCHITECTURE

05 | APPROACH

06 | RESULTS

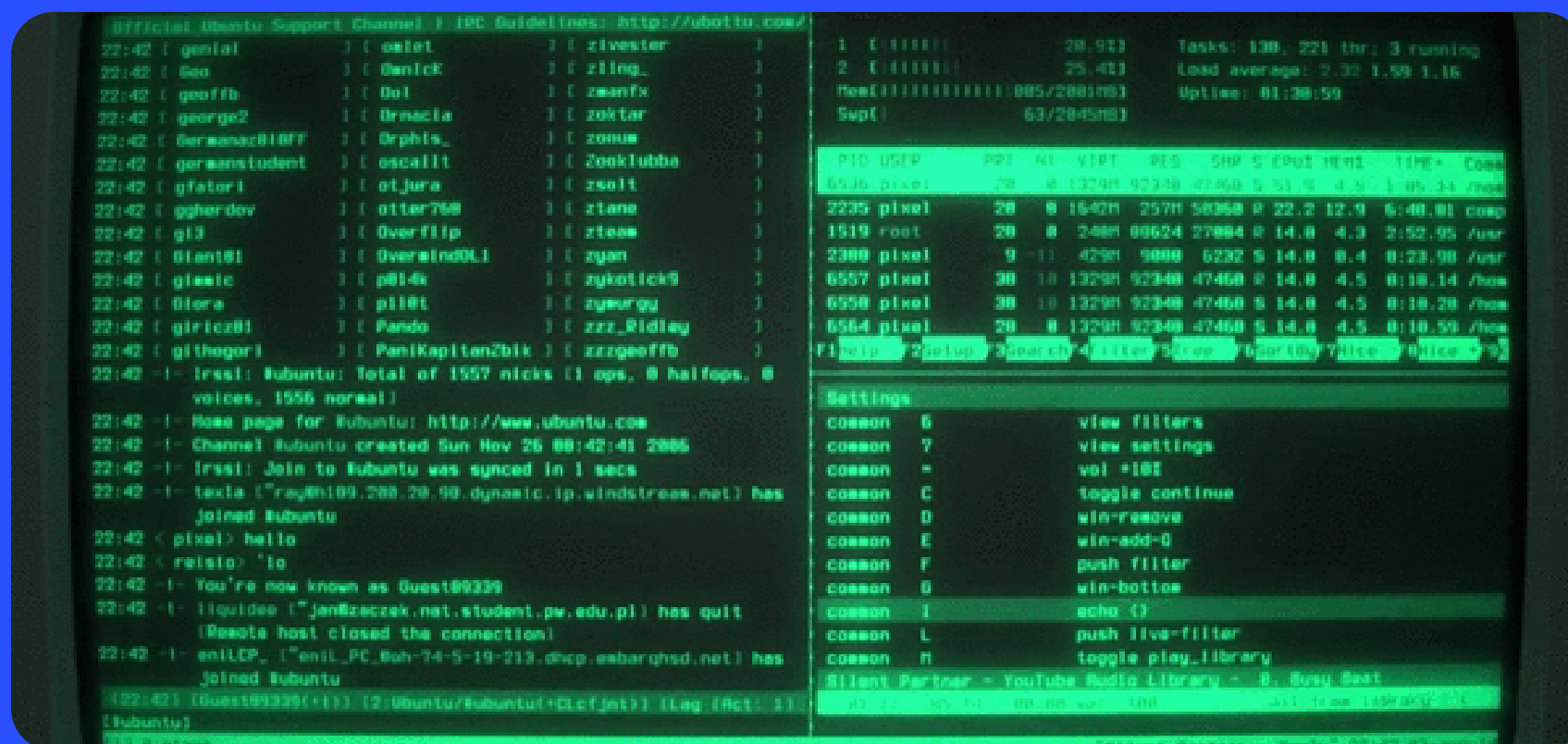
07 | LIVE DEMO

08 | CHALLENGES,
LEARNINGS &
FUTURE SCOPE



ANOMALYZE

PROBLEM STATEMENT



WHAT ARE WE SOLVING

- **Massive Logs (TBs/day):** Distributed systems, like Hadoop, generate terabytes of logs every day, capturing details on millions of operations and system behaviors. Analyzing this enormous volume of data manually or with inadequate tools becomes overwhelming.
- **Hard to Monitor Manually:** Log files are often unstructured, noisy, and stored in large, scattered clusters. Sifting through this data manually to identify issues or anomalies is time-consuming, error-prone, and unsustainable for modern IT environments.
- **Delayed Anomaly Detection → Downtime & Losses:** Without real-time monitoring and anomaly detection, critical issues may go unnoticed for hours or even days. This leads to prolonged downtime, reduced system reliability, and significant operational and financial losses for businesses reliant on distributed systems.



MOTIVATION

- Massive volumes of log data generated in distributed systems make manual monitoring and analysis impractical.
- Undetected anomalies in Hadoop and large-scale systems can lead to downtime, performance degradation, and operational failures.
- Real-time automated anomaly detection is essential for maintaining system reliability, scalability, and efficiency in DevOps environments.
- Industry-scale platforms such as those used by Amazon, Google, and Facebook rely heavily on intelligent monitoring systems to ensure high availability.
- This project focuses on building a scalable and efficient pipeline for automated log analysis and anomaly detection.



shutterstock.com · 2641239033



KEY FEATURES

REAL TIME LOG INGESTION

Continuously stream and process massive volumes of distributed system logs via Kafka.

FEATURE EXTRACTION

Convert raw logs into structured information with enriched fields like timestamps, components, and error types for easier analysis.

ANOMALY DETECTION

Detect critical issues using:

Error spikes: Identify unexpected bursts in error or warning logs

Traffic anomalies: Spot unusual patterns or deviations in system behavior

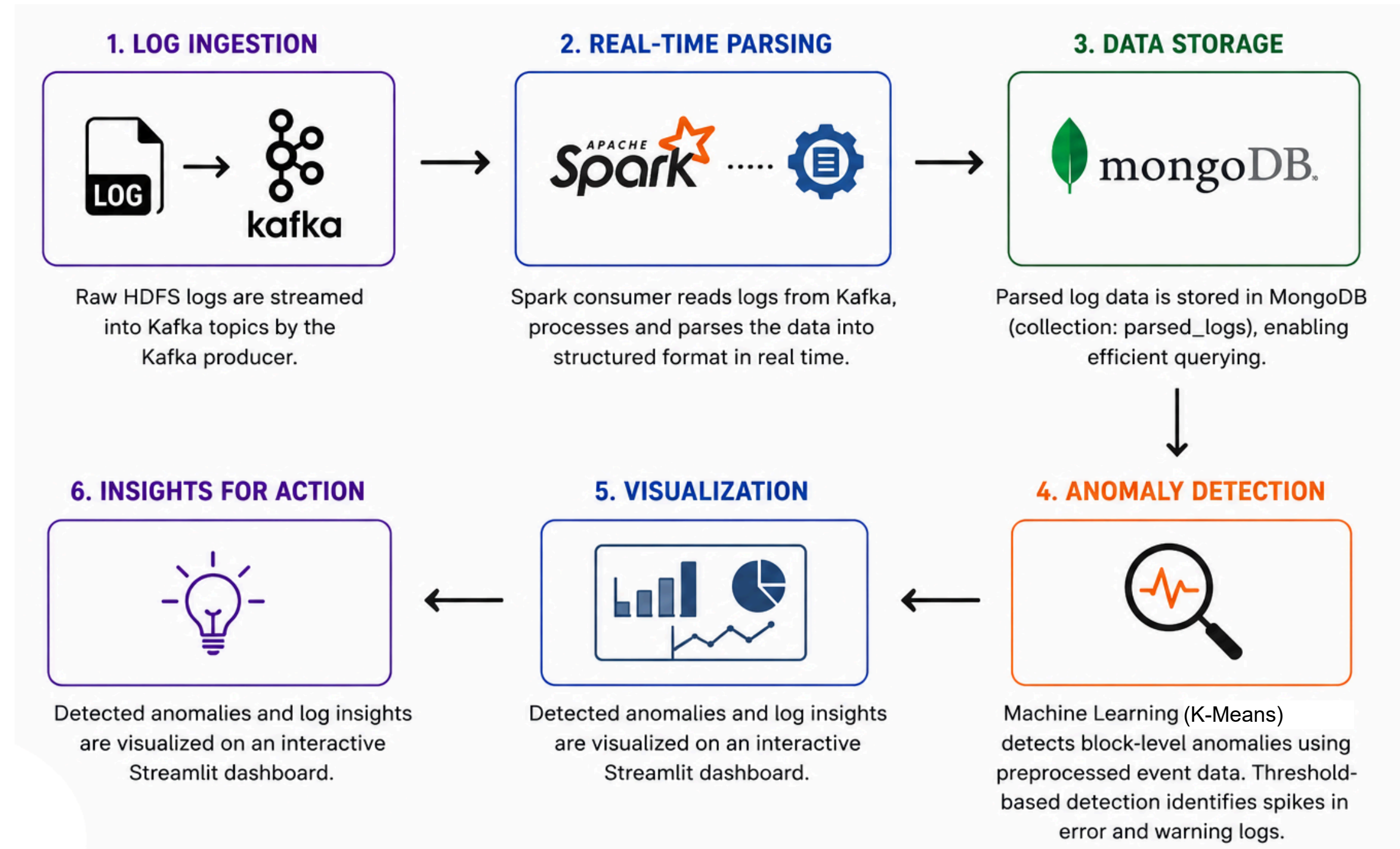
VISUALIZATION

Interactive Dashboard built using ReactJs offering detailed insights, trends, and anomaly summaries for swift decision-making

FAILURES

Highlight critical failures in specific components or operations

ARCHITECTURE / PROJECT FLOW



ANOMALY DETECTION APPROACH

1. Statistical Thresholding:

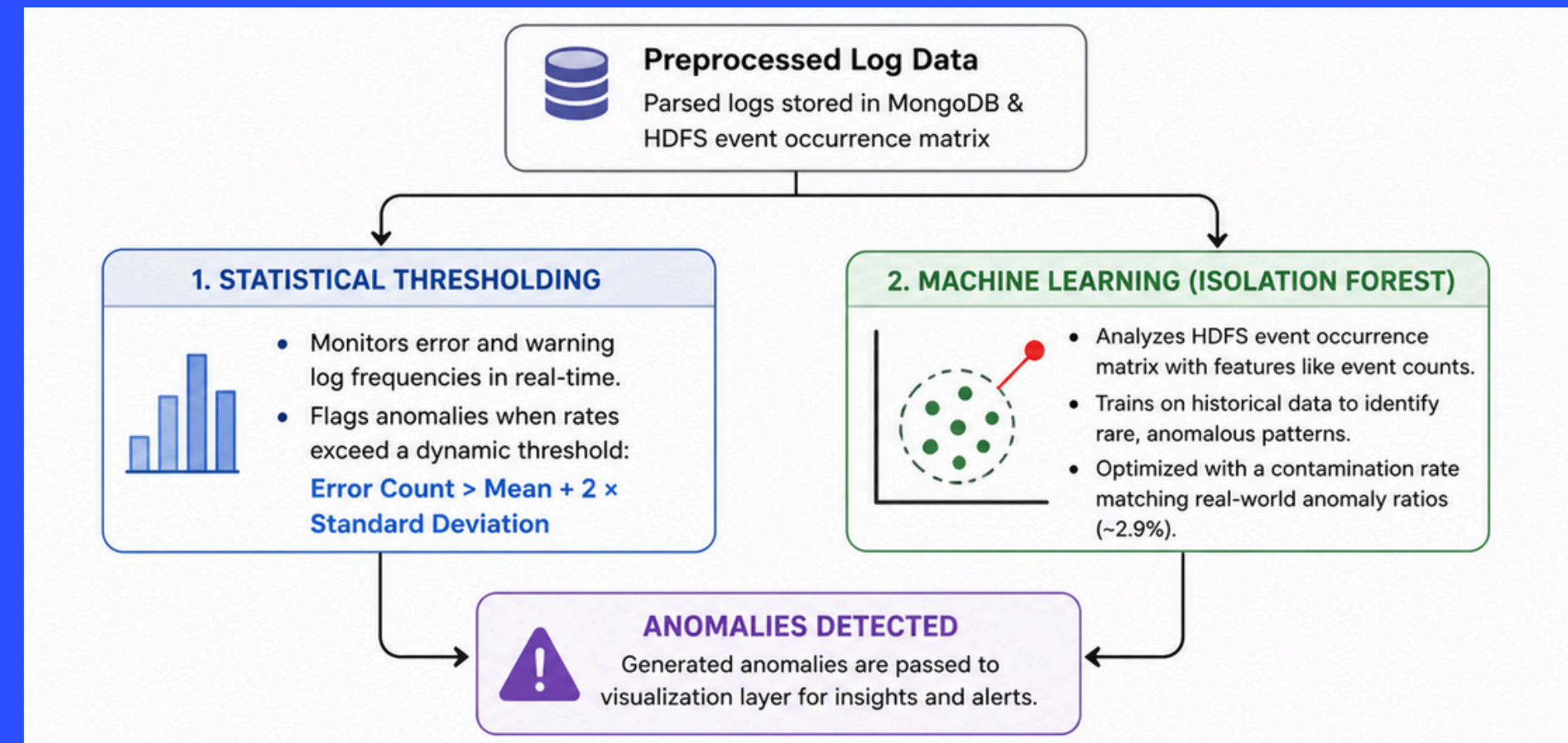
- Monitors error and warning log frequencies in real-time.
- Flags anomalies when rates exceed a dynamic threshold:
- $\text{Error Count} > \text{Mean} + 2 \times \text{Standard Deviation}$.

2. Machine Learning (Isolation Forest):

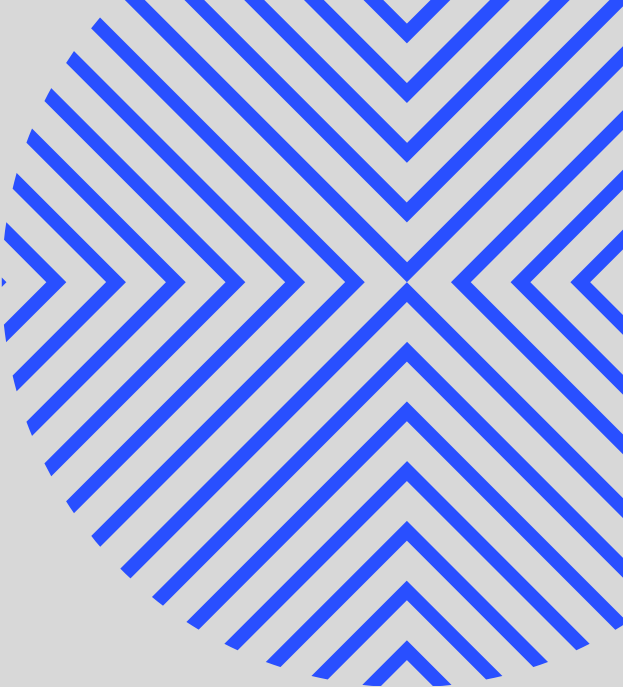
- Analyzes HDFS event occurrence matrix with features like event counts.
- Trains on historical data to identify rare, anomalous patterns.
- Optimized with a contamination rate matching real-world anomaly ratios (~2.9%).

3. Dual Approach Benefits:

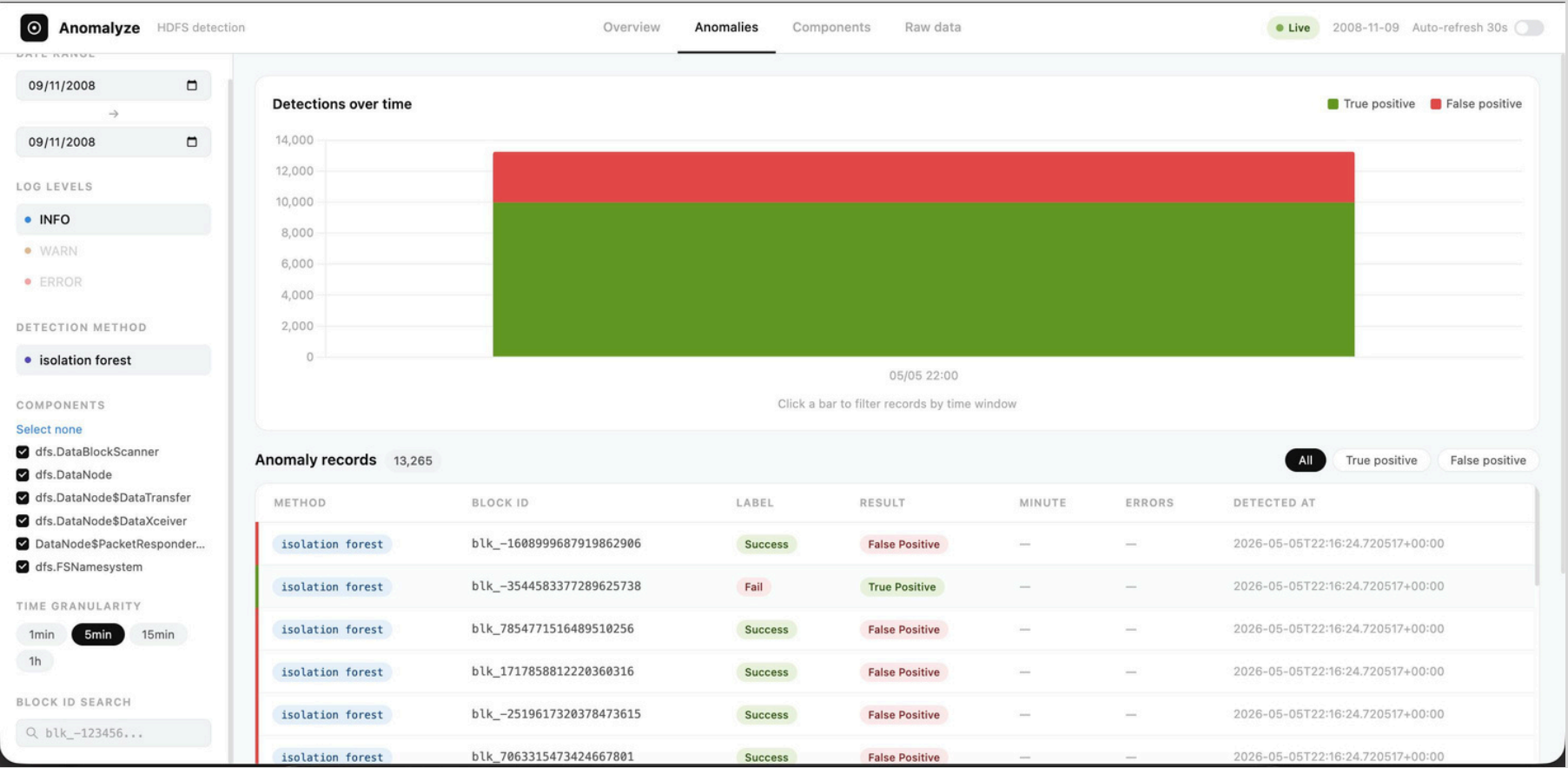
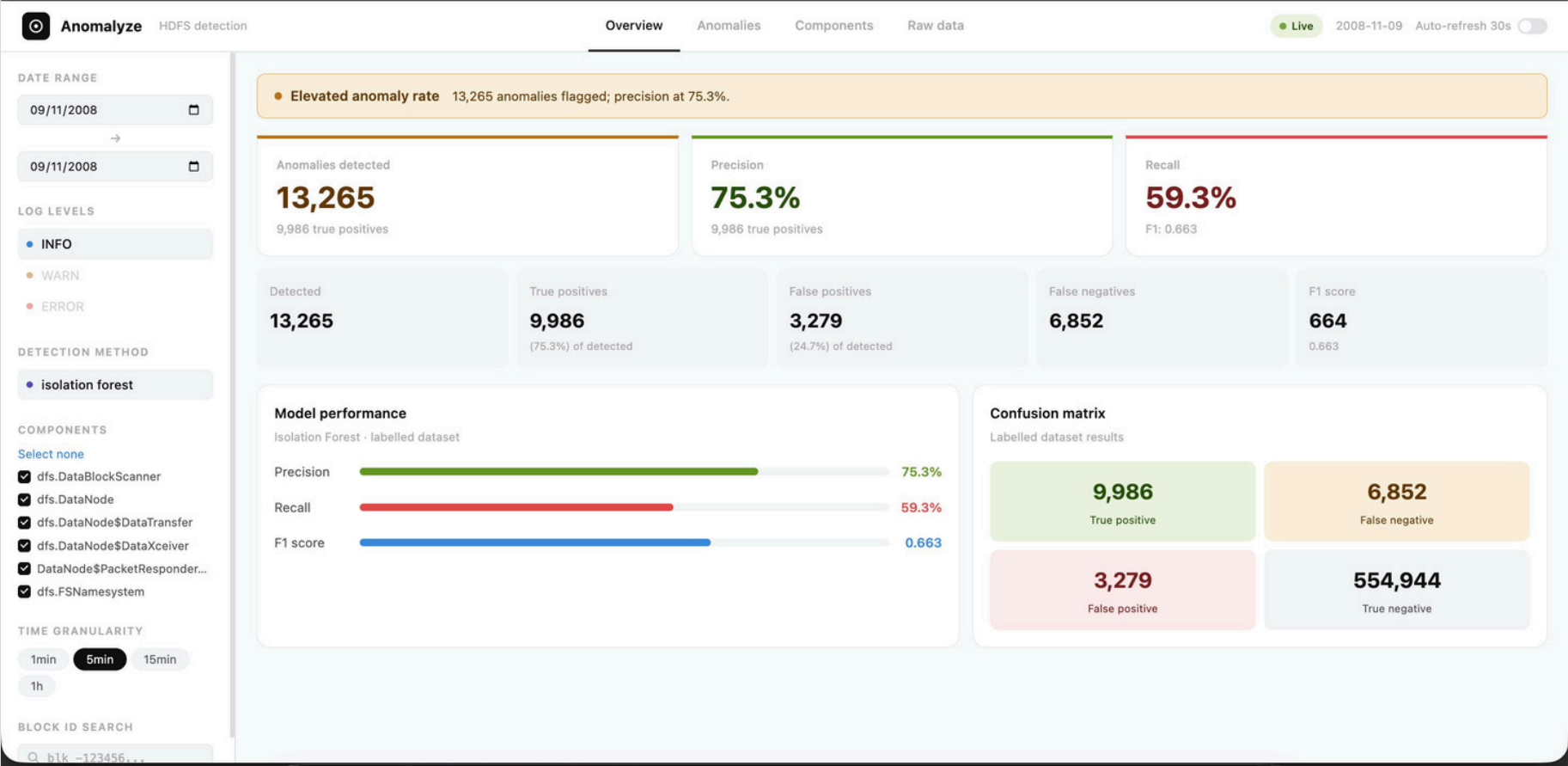
- Statistical Detection: Quick and effective for error spikes.
- ML Detection: Captures subtle, non-linear anomalies in highly complex data.



ANOMALYZE IN ACTION



LIVE DEMO AND RESULTS



CHALLENGES & LEARNINGS

Challenge	Solution	Learning
Integrating Kafka with Spark Streaming	We solve it with a kafka-python producer pushing HDFS log lines to the hdfs-logs topic, and a PySpark Structured Streaming consumer (spark.readStream.format("kafka")+foreachBatch) that parses each batch and writes the results into MongoDB.	Decoupling ingestion and processing enables scalable, fault-tolerant real-time pipelines with reliable data flow.
Handling inconsistent and unstructured log formats during parsing	We handle inconsistent raw logs with a shared regex-based parser (parse_hdfs_raw_line) that normalizes matching HDFS lines into structured fields and safely skips unmatched records by returning None.	Robust data pipelines must normalize messy real-world inputs and gracefully handle invalid records without breaking the system.
Choosing the best anomaly detection / ML algorithm	We use K-Me because it detects rare, abnormal log-behavior patterns efficiently without requiring labeled anomaly data, which fits large HDFS log streams well.	Choosing the right model based on data constraints is critical for building effective real-world anomaly detection systems.

FUTURE SCOPE



Add real-time alerting and automated incident response so anomalies trigger notifications and remediation workflows immediately.

Expand the pipeline to support multiple log sources and richer models



Productionize the platform with scalable deployment, model monitoring, and feedback loops



THANK YOU !!

Harindham Sharma

Aman Kumar

Ananya Agarwal

Gurleen Kaur

Aditya Kolluru Naidu